

- Entriamo in un ciclo **while**, continuando finché la coda non è vuota:
  - rimuoviamo una cella  $(r, c)$  dalla coda;
  - ricoloriamo la cella corrente con il nuovo colore *color*;
  - per ciascuna delle quattro direzioni definite in *dr* e *dc*, calcoliamo le coordinate  $(nr, nc)$  della cella adiacente;
  - se la cella  $(nr, nc)$  è all'interno dei limiti della matrice e ha il colore *oldcolor*, la inseriamo nella coda *Q*.
- Terminiamo il processo quando tutte le celle raggiungibili con colore iniziale *oldcolor* sono state ricolorate.

L'algoritmo include un controllo iniziale per evitare operazioni inutili: se il colore *color* è uguale al colore iniziale *oldcolor*, non viene eseguito alcun riempimento.

---

```

int floodfill(int[][] M, int n, int x, int y, int color)
int[] dr = [-1, 0, +1, 0]                                % Mosse possibili sulle righe
int[] dc = [0, -1, 0, +1]                                % Mosse possibili sulle colonne
int oldcolor = M[x][y]
if color ≠ oldcolor then
  QUEUE Q = Queue()
  Q.enqueue(x, y)
  M[x][y] = color
  while not Q.isEmpty() do                                % Riga, colonna della cella visitata correntemente
    int, int r, c = Q.dequeue()
    for i = 1 to 4 do
      nr = r + dr[i]                                     % Nuova riga
      nc = c + dc[i]                                     % Nuova colonna
      if 1 ≤ nr ≤ n and 1 ≤ nc ≤ n and M[nr][nc] = oldcolor then
        M[nr][nc] = color
        Q.enqueue(nr, nc)

```

---

L'algoritmo è corretto poiché ogni cella che deve essere ricolorata viene visitata una sola volta. Questo è garantito dal fatto che le celle vengono aggiunte alla coda solo se il loro colore è ancora *oldcolor*. Una volta ricolorata, non sarà più inserita in coda. Il controllo iniziale assicura che nessuna operazione venga eseguita se il colore desiderato è già uguale al colore della cella di partenza.

La complessità dell'algoritmo è determinata dalla visita di un grafo con  $n^2$  nodi, corrispondenti alle celle della matrice, e al massimo  $4n^2$  archi, corrispondenti alle connessioni tra celle adiacenti. La complessità temporale è quindi  $O(n^2)$ , poiché ogni nodo viene visitato una sola volta.

## Esercizio B1 – Serie aritmetiche – Punti $\geq 8$

Non è necessaria una particolare tecnica algoritmica, solo buon senso.

---

```

countArithmetic(int[] A, int n)
int series = 0
int lastDiff = +∞
for i = 2 to n do
  int currentDiff = A[i] - A[i - 1]
  if currentDiff ≠ lastDiff then
    lastDiff = currentDiff
    count = 1
  else
    count = count + 1
    if count == 2 then
      series = series + 1
return series

```

---

### 1. Inizializzazione delle variabili:

- *series*: mantiene il conteggio del numero di sottovettori aritmetici massimali trovati.

- *lastDiff*: memorizza la differenza tra i valori consecutivi nel sottovettore corrente. Inizialmente è impostato a  $+\infty$  per distinguere il primo conteggio.
- *count*: tiene traccia del numero di differenze consecutive uguali nel sottovettore corrente.

2. **Ciclo principale:** Si itera attraverso l'array  $A$  a partire dalla seconda posizione ( $i = 2$ ).

- Per ogni coppia consecutiva di elementi  $A[i-1]$  e  $A[i]$ , si calcola la differenza  $currentDiff = A[i] - A[i-1]$
- **Caso 1: La differenza cambia** ( $currentDiff \neq last$ ):
  - Si aggiorna  $last = currentDiff$ .
  - Si reimposta  $count = 1$ , poiché il nuovo sottovettore inizia con una sola differenza calcolata.
- **Caso 2: La differenza è uguale a quella precedente** ( $currentDiff = last$ ):
  - Si incrementa  $count$  di 1.
  - Se  $count = 2$ , significa che abbiamo identificato un sottovettore aritmetico (almeno tre elementi consecutivi con la stessa differenza). Si incrementa *series* di 1; eventuali valori successivi che aumentano la lunghezza non verranno considerati.

3. **Risultato:** Alla fine del ciclo, la variabile *series* contiene il numero totale di sottovettori aritmetici massimali trovati, che viene restituito come output.

L'algoritmo scorre l'array  $A$  una sola volta, quindi la complessità temporale è  $O(n)$ .

## Esercizio B2 – Radice-foglia – Punti $\geq 10$

L'algoritmo `allpath()` utilizza un approccio ricorsivo per esplorare tutti i cammini radice-foglia di un albero binario. La funzione principale inizializza una struttura *stack* per memorizzare il cammino corrente e avvia la funzione ricorsiva ausiliaria `apRec`, passando la radice dell'albero come parametro iniziale.

La funzione `apRec()` si occupa di visitare ogni nodo dell'albero. Quando viene chiamata su un nodo  $t$ , aggiunge il nodo corrente alla pila *stack*, che rappresenta il cammino parziale dalla radice al nodo corrente. Se il nodo  $t$  è una foglia, ovvero non ha figli né a sinistra né a destra, il contenuto della pila *stack*, che rappresenta ora il cammino completo dalla radice alla foglia, viene stampato. In seguito, la funzione chiama ricorsivamente se stessa sui figli sinistro e destro del nodo corrente, continuando l'esplorazione dell'albero.

Dopo aver visitato i sottoalberi del nodo corrente, la funzione esegue un'operazione di backtracking rimuovendo il nodo corrente dalla pila *stack*. Questo garantisce che la struttura della pila rifletta correttamente il cammino attivo quando si ritorna al nodo padre. Questo meccanismo di backtracking è fondamentale per assicurare che ogni cammino radice-foglia venga stampato esattamente una volta.

---

```
allpath(TREE T)
```

---

```
    STACK stack = Stack()
    apRec(T, stack)
```

---



---

```
apRec(TREE t, STACK stack)
```

---

```
    if t ≠ nil then
        stack.push(t)
        if t.left() == nil and t.right() == nil then
            | print stack                                % Raggiunta una foglia, stampo il cammino
        apRec(t.left(), stack)
        apRec(t.right(), stack)
        stack.pop()
```

---

L'algoritmo è corretto poiché visita ogni nodo dell'albero esattamente una volta e, per ogni foglia raggiunta, stampa il cammino corrispondente dalla radice a quella foglia. Il backtracking permette di gestire correttamente il cammino durante la visita ricorsiva, evitando conflitti tra percorsi diversi.

Dal punto di vista computazionale, l'algoritmo visita ogni nodo una sola volta, ma la stampa dei cammini contribuisce in modo significativo alla complessità complessiva. È utile considerare diversi casi per comprendere meglio l'impatto della struttura dell'albero sulla complessità:

- **Albero degenerato:** Se l'albero è degenerato, ossia una lista collegata, esiste un unico cammino radice-foglia, poiché c'è una sola foglia. In questo caso, l'algoritmo ha un costo totale di  $O(n)$ , dove  $n$  è il numero di nodi dell'albero.

- **Albero bilanciato:** In un albero completo bilanciato, esistono approssimativamente  $n/2$  foglie, ciascuna delle quali corrisponde a un cammino dalla radice alla foglia. Poiché la profondità di un albero bilanciato è  $O(\log n)$ , ogni cammino ha lunghezza  $O(\log n)$ . Il costo complessivo dell'algoritmo in questo caso è quindi  $O(n \log n)$ , dato che la stampa di  $n/2$  cammini di lunghezza  $O(\log n)$  domina il costo del semplice attraversamento dell'albero.
- **Caso generale:** È possibile costruire un albero in cui la stampa raggiunge una complessità totale di  $O(n^2)$ . Consideriamo un albero composto da  $n/2$  nodi disposti a lista collegata, con in fondo a questa lista un sottoalbero completo con  $n/2$  nodi. In questa configurazione, ci sono circa  $n/4$  foglie, e ciascun cammino ha una lunghezza di  $n/2 + O(\log n)$ , poiché include il percorso nella lista e la profondità del sottoalbero completo. La stampa di  $n/4$  cammini di lunghezza  $n/2 + O(\log n)$  produce una complessità totale di  $O(n^2)$ .

L'algoritmo non può impiegare più di  $O(n^2)$ , perché mal che vada deve stampare un cammino di lunghezza  $O(n)$  per ognuno degli  $n$  nodi. Identificare tuttavia un albero in cui questo accade è interessante, perché mostra che tale complessità può essere raggiunta.

### Esercizio B3 – Quante parentesi – Punti $\geq 12$

L'algoritmo calcola il numero di parentesizzazioni corrette utilizzando la programmazione dinamica, sfruttando la relazione tra le parentesizzazioni e i numeri di Catalan. Una parentesizzazione corretta può essere costruita seguendo la logica:

- Si inizia con una parentesi aperta.
- La parentesi aperta è seguita da una parentesizzazione valida di lunghezza  $i$  (con  $i$  compreso tra 0 e  $k-1$ ).
- La parentesi chiusa completa la parentesizzazione, lasciando  $k-i-1$  coppie da completare.

Le parentesizzazioni di lunghezza  $i$  e quelle di lunghezze  $ki-i-1$  si combinano insieme; quindi questi numeri vanno moltiplicati insieme.

La formula è quindi:

$$DP[k] = \begin{cases} 1 & k = 0 \text{ or } k = 1 \\ \sum_{i=0}^{k-1} DP[i] \cdot DP[k-i-1] & k > 1 \end{cases}$$

---

countPars(int n)

---

```
int [] DP = new int[0...n]
DP[0] = DP[1] = 1
for k = 2 to n do
    DP[k] = 0
    for i = 0 to k-1 do
        DP[k] = DP[k] + DP[i] * DP[k-i-1]
return DP[n]
```

---

L'output per  $n$  che va da 1 a 10 è quindi [1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796].

La complessità è  $O(n^2)$ , poiché per ogni  $n$  calcoliamo una somma che coinvolge  $n-1$  elementi.